Game Plaza - Database Schema Design

Database: game_plaza

Table 1: users
```sql
CREATE TABLE users (
 id INT PRIMARY KEY AUTO_INCREMENT,
 username VARCHAR(50) UNIQUE NOT NULL,
 email VARCHAR(100) UNIQUE NOT NULL,
 password_hash VARCHAR(255) NOT NULL,
 bio TEXT,
 skill_level ENUM('Beginner', 'Intermediate', 'Advanced', 'Pro') DEFAULT
'Beginner',
 availability VARCHAR(100),
 profile_created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

Table 2: games
```sql
CREATE TABLE games (
 id INT PRIMARY KEY AUTO_INCREMENT,
 name VARCHAR(100) NOT NULL UNIQUE,
 genre VARCHAR(50),
 developer VARCHAR(100),
 release_year INT,
 is_multiplayer BOOLEAN DEFAULT TRUE
);
```

Table 3: user_games (Many-to-Many)
```sql
CREATE TABLE user_games (
 id INT PRIMARY KEY AUTO_INCREMENT,
 user_id INT NOT NULL,
 game_id INT NOT NULL,
 platform ENUM('Steam', 'Epic Games', 'Blizzard', 'Riot Games', 'Other') DEFAULT
'Steam',
 playtime_hours INT DEFAULT 0,
 added_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
 FOREIGN KEY (game_id) REFERENCES games(id) ON DELETE CASCADE,
 UNIQUE KEY unique_user_game (user_id, game_id)
);
```

Table 4: user_preferences
```sql
CREATE TABLE user_preferences (
 id INT PRIMARY KEY AUTO_INCREMENT,
 user_id INT NOT NULL UNIQUE,
 preferred_genres VARCHAR(200),
 preferred_playstyle VARCHAR(100),
 competitive_preference ENUM('Casual', 'Competitive', 'Mixed') DEFAULT 'Mixed',
 team_size_preference ENUM('Solo', 'Duo', 'Squad', 'Large') DEFAULT 'Squad',
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
 FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

Table 5: matches
```sql
CREATE TABLE matches (
 id INT PRIMARY KEY AUTO_INCREMENT,
 name VARCHAR(100) NOT NULL,
 game_id INT NOT NULL,
 creator_id INT NOT NULL,
 match_type ENUM('Casual', 'Tournament', 'Ranked') DEFAULT 'Casual',
 max_participants INT DEFAULT 4,
 status ENUM('Pending', 'Active', 'Completed', 'Cancelled') DEFAULT 'Pending',
 scheduled_time DATETIME,
 description TEXT,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
 FOREIGN KEY (game_id) REFERENCES games(id),
 FOREIGN KEY (creator_id) REFERENCES users(id) ON DELETE CASCADE
);
```

Table 6: match_participants
```sql
CREATE TABLE match_participants (
 id INT PRIMARY KEY AUTO_INCREMENT,
 match_id INT NOT NULL,
 user_id INT NOT NULL,
 joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 status ENUM('Pending', 'Confirmed', 'Declined', 'Left') DEFAULT 'Pending',
 FOREIGN KEY (match_id) REFERENCES matches(id) ON DELETE CASCADE,
 FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
 UNIQUE KEY unique_match_user (match_id, user_id)
```

```
);
```

Table 7: likes_follows
```sql
CREATE TABLE likes_follows (
 id INT PRIMARY KEY AUTO_INCREMENT,
 follower_id INT NOT NULL,
 followee_id INT NOT NULL,
 action_type ENUM('Like', 'Follow') DEFAULT 'Follow',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 FOREIGN KEY (follower_id) REFERENCES users(id) ON DELETE CASCADE,
 FOREIGN KEY (followee_id) REFERENCES users(id) ON DELETE CASCADE,
 UNIQUE KEY unique_follow (follower_id, followee_id)
);
```

Table 8: compatibility_scores (Optional - Cache)
```sql
CREATE TABLE compatibility_scores (
 id INT PRIMARY KEY AUTO_INCREMENT,
 user_a_id INT NOT NULL,
 user_b_id INT NOT NULL,
 score FLOAT DEFAULT 0,
 shared_games INT DEFAULT 0,
 calculated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
 FOREIGN KEY (user_a_id) REFERENCES users(id) ON DELETE CASCADE,
 FOREIGN KEY (user_b_id) REFERENCES users(id) ON DELETE CASCADE,
 UNIQUE KEY unique_compatibility (user_a_id, user_b_id)
);
```

KEY RELATIONSHIPS

```
users (1) ----< (Many) user_games >---- (1) games
users (1) ----< (Many) matches (creator)
users (1) ----< (Many) match_participants >---- (1) matches
users (1) ----< (Many) likes_follows >---- (1) users (followee)
users (1) ----< (1) user_preferences
```

INDEXES (Performance)

```sql
CREATE INDEX idx_user_games_user_id ON user_games(user_id);
CREATE INDEX idx_user_games_game_id ON user_games(game_id);
CREATE INDEX idx_matches_creator_id ON matches(creator_id);
CREATE INDEX idx_matches_game_id ON matches(game_id);
CREATE INDEX idx_match_participants_match_id ON match_participants(match_id);
CREATE INDEX idx_match_participants_user_id ON match_participants(user_id);
```

SAMPLE DATA CONCEPT

Sample Games:
- League of Legends (Riot Games, MOBA)
- Counter-Strike 2 (Steam, FPS)
- Valorant (Riot Games, FPS)
- Dota 2 (Steam, MOBA)
- Minecraft (Multi-platform, Sandbox)

Sample Users:
- 5-10 test users with different skill levels and preferences
- Users with overlapping games for matchmaking testing

Sample Matches:
- 3-5 test matches in pending/active state

NOTES

- **Normalization**: 3NF - No partial dependencies
- **Primary Keys**: All tables have INT auto-increment
- **Foreign Keys**: All properly constrained
- **Timestamps**: Track creation and updates
- **Enums**: Restrict values to valid options
- **Indexes**: On frequently queried columns